

From Functional Builders to Broken Pits

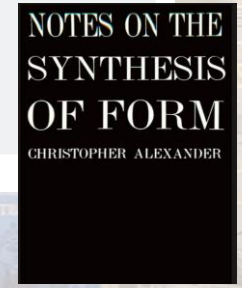
Hans Kruse

Please questions at the end, thank you

KVK

A Book

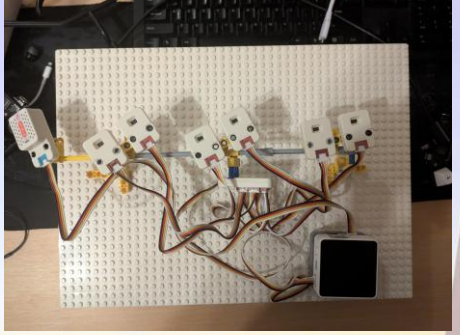
```java
new Book(
    "Notes on the Synthesis of Form",
    "Christopher Alexander",
    "9780674627512"
);
```



- Argument order matters
- Swapped values still compile
- No defaults values or named parameters in Java

KVK



Software Engineer

HANS KRUSE






KVK

KVK, Trust in Trade

- KVK, the Dutch Chamber of Commerce
- Manages the Dutch Business Register
- Founded in 1803 in Rotterdam
- Provides trusted business identity data
- Issues export documents

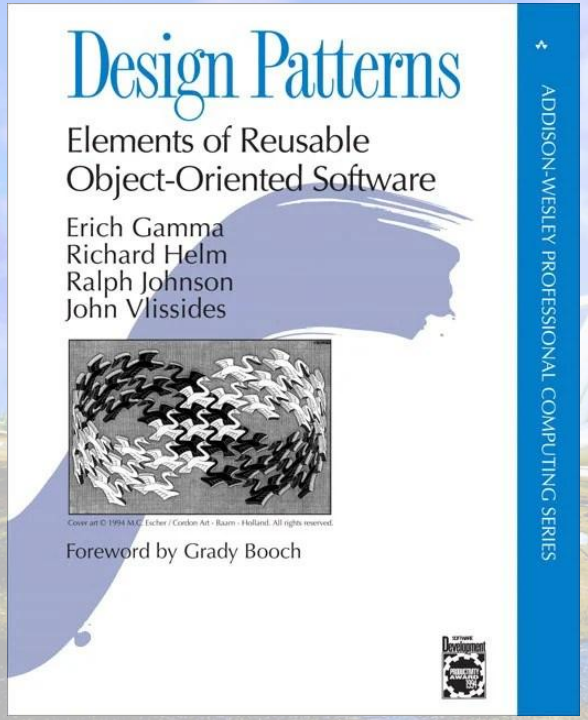
Argument soup

```
new Book(
    title,
    subtitle,
    author,
    isbn,
    publisher,
    publicationYear,
    language,
    edition,
    pageCount,
    deweyClassification,
    libraryOfCongressNumber,
    translator,
    series,
    tags,
    rating,
    internalId
);
```

- More to get wrong
- Constructors overloads?
- C# has
  - named parameters
  - defaults
- Java does not

KVK

Design patterns



- Patterns gave us names
- Names gave us shared understanding
- Shared understanding lets us express increasingly specific constraints
- The Elasticsearch Java client does exactly that in its API design

KVK

Builder

```java
import lombok.Builder;

@Builder
public record Book(String title,String author,String isbn) {
}
```

```java
Book book = Book.builder()
    .title("Effective Java")
    .author("Joshua Bloch")
    .isbn("978-0134685991")
    .build();
```

KVK

Elasticsearch-java client

- Generated from API specifications
- Thin client over REST
- No longer "half a server"
- Similar API across languages
- Easier to keep in sync with Elasticsearch
- Has (functional) builders…

KVK

Functional Builder in elasticsearch-java

- Builder pattern + lambdas
- No builder variable
- Strong typing
- Readable nested structures
- Closely matches the JSON

```
SearchRequest request = SearchRequest.of(s -> s
    .index("books")
    .query(q -> q
        .match(m -> m
            .field("title")
            .query("java")
        )
    )
);
```

KVK

Initial DIY with Lombok

```java
import java.util.function.UnaryOperator;

import lombok.Builder;

@Builder
public record Book(String title,String author,String isbn) {

    public static Book of(UnaryOperator<BookBuilder> fn) {
        return fn.apply(Book.builder()).build();
    }

    public static void main(String[] args) {
        Book book = Book.of(b -> b
            .title("Effective Java")
            .author("Joshua Bloch")
            .isbn("978-0134685991")
        );

        System.out.println(book);
    }
}
```

- Recreating
- UnaryOperator<Builder>
- Close but not quite

KVK

More the Elastic way

```java
import lombok.Builder;

interface ObjectBuilder<T> {
    T build();
}

@Builder
public record Book(
    String title,
    String author,
    String isbn
) {
    public static class BookBuilder
            implements ObjectBuilder<Book> {
    }
}
```

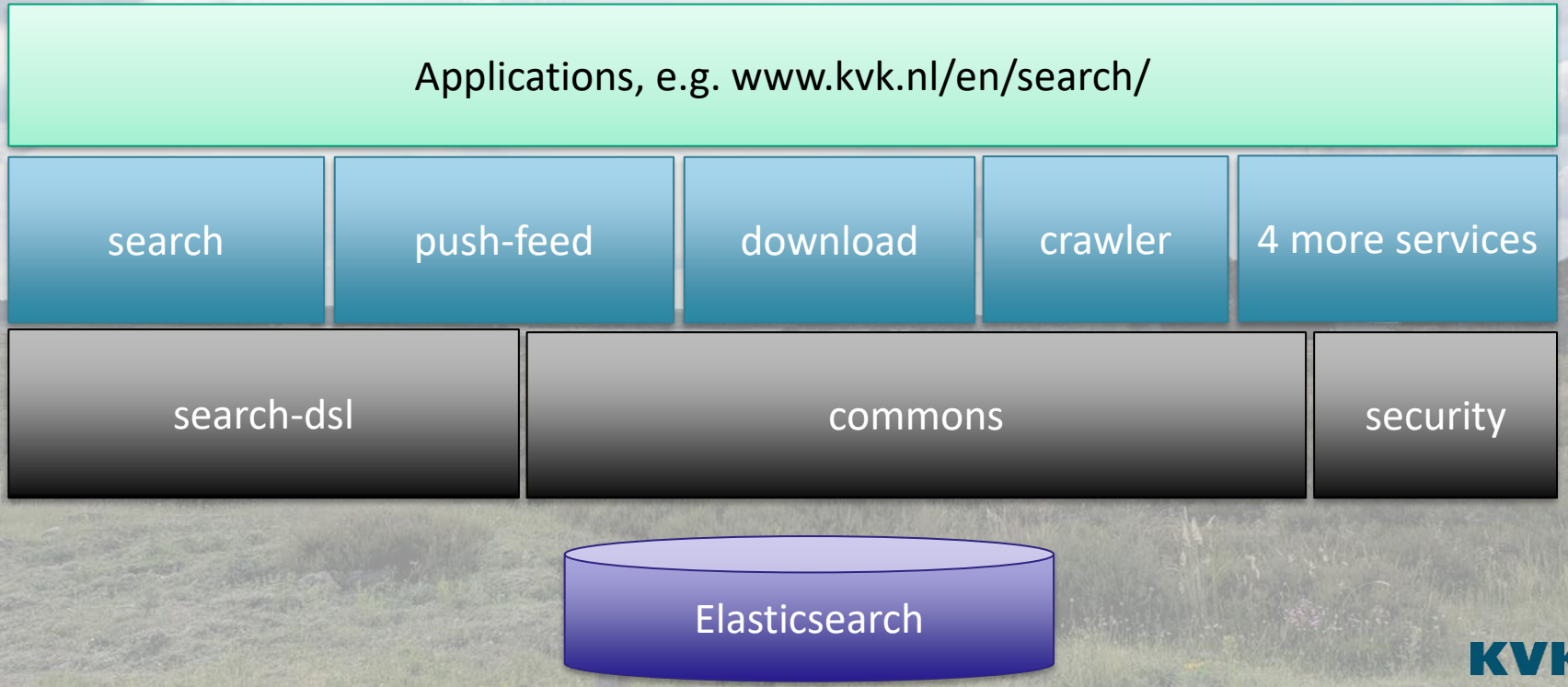
```java
import java.util.function.Function;

public static Book of(
    Function<BookBuilder, ObjectBuilder<Book>> fn
) {
    return fn.apply(Book.builder())
             .build();
}

Book book = Book.of(b -> b
    .title("Effective Java")
    .author("Joshua Bloch")
    .isbn("978-0134685991")
);
```

KVK

High level Enterprise-Search overview



Migration roadmap

1. Prepare

- Upgrade Elasticsearch 7.x and HLRC
- Resolve deprecations
- Enable compatibility mode

2. Migrate

- Upgrade to Elasticsearch 8
- ***Replace HLRC with elasticsearch-java***

3. Modernise

- Migrate from ECE to ECK
- Upgrade to Elasticsearch 8.19

4. Prepare for 9.x

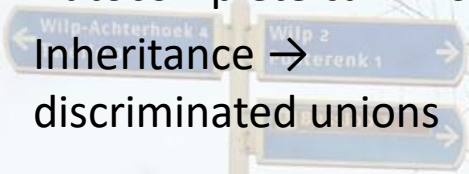
- Enable 9.x compatibility mode
- Start the Elasticsearch 9 migration

KVK

HLC to Elasticsearch-java

- Where to start?
- Immature documentation
- Autocomplete can mislead you
- Inheritance → discriminated unions

- Return Query, follow compiler errors
- Functional builders: adopt or adapt?



KVK

Details

Query model

- One search, one result type
- Mixed indexes? Use JsonObject
- multi_match works differently
- Highlighting works differently

API changes

- Refresh constant missing
- Bool query construction changed
- Per-field boosting syntax changed
- Index boosts now use Double
- * methods are usually generated helpers

Testing

- Query is less transparent
- JSON extraction was harder
- Generated JSON changed
- System tests needed updates

KVK

Elasticsearch-java 8.15.5 to 8.19.22

8.16

- AutoCloseable support.
- **required shards field**
- indexFilter in openPointIntTime

8.17

- allowPartialSearchResults for openPointInTime

8.18

- SearchRequest.indicesBoost changed from *List<Map<String, Double>>* to *List<NamedValue<Double>>*.

8.19.0

- HealthStatus added *Unknown* and *Unavailable*
- handle these statuses explicitly.
- Extra required fields

8.19.0 to 8.19.22

- bugfixes
- Non relevant changes.

KVK

Copilot the Pitmaster

What happened?

- New client: **8.19.19**
- Cluster: **8.15.0**
- Opening a PIT succeeded
- The client still threw an exception, `_shards` field not available

Workaround

- Pitmaster GitHub Copilot spotted a way out
- The PIT id was already available
- Copilot extracted the PIT id from the exception and carried on

Disclaimer

- Not recommended by Elastic
- Not something we'd normally advocate
- Too funny not to share with Elastic, and you. we wondered if they could fix it?
- Elastic said no

KVK

Vibe code: overview

```java
    public String getPit(final String alias,
                         final int keepAlive) throws IOException {
        if (keepAlive <= 0) {
            throw new IllegalArgumentException(KEEP_ALIVE_MUST_BE_GREATER_THAN_0);
        }
        final var mostRecentIndexUnderAlias = clusterService.getMostRecentIndexUnderAliasName(alias);
        final var openPitRequest = new OpenPointInTimeRequest.Builder()
                .index(mostRecentIndexUnderAlias)
                .keepAlive(timeBuilder -> timeBuilder.time(keepAlive + "m"))
                .build();
        try {
            return client.openPointInTime(openPitRequest).id();
        } catch (final TransportException transportException) {
            if (isMissingShardsOnPitResponse(transportException)) {
                return extractPitIdFromRawBody(transportException);
            }
            throw transportException;
        }
    }
```

KVK

VibeCode: missing _shards?

```java
    /**
     * Detects the specific case where the Elasticsearch server (pre-8.16) returns a PIT response
     * without the 'shards' field that the 8.16+ Java client model requires.
     * Uses the TransportException's endpoint ID to scope the check precisely to openPointInTime calls.
     * Package-private for unit testing.
     */
    boolean isMissingShardsOnPitResponse(final TransportException transportException) {
        if (!OPEN_PIT_ENDPOINT_ID.equals(transportException.endpointId())) {
            return false;
        }
        final var cause = transportException.getCause();
        if (!(cause instanceof MissingRequiredPropertyException mrpe)) {
            return false;
        }
        // Property name can be either "shards" or "OpenPointInTimeResponse.shards"
        final var propName = mrpe.getPropertyName();
        return PIT_MISSING_SHARDS_PROPERTY.equals(propName) || propName.endsWith("." + PIT_MISSING_SHARDS_PROPERTY);
    }
```

KVK

Vibe code: Recover from exception

```java
    /**
     * Extracts the PIT 'id' field from the raw transport response body.
     * Used as fallback when the server (pre-8.16) omits the 'shards' field.
     * Package-private for unit testing.
     */
    String extractPitIdFromRawBody(final TransportException transportException) throws IOException {
        final var response = transportException.response();
        if (isNull(response)) {
            throw new IOException("PIT response body unavailable after MissingRequiredPropertyException for 'shards'", transportException);
        }
        try (var repeatableResponse = RepeatableBodyResponse.of(response)) {
            final var body = repeatableResponse.body();
            if (isNull(body)) {
                throw new IOException("PIT response body is null after MissingRequiredPropertyException for 'shards'", transportException);
            }
            try (var bodyStream = body.asInputStream()) {
                final var rawBody = new String(bodyStream.readAllBytes(), StandardCharsets.UTF_8);
                final var idNode = FALLBACK_OBJECT_MAPPER.readTree(rawBody).get("id");
                if (isNull(idNode) || idNode.isNull()) {
                    throw new IOException("PIT response missing 'id' field. Raw body: " + rawBody);
                }
                log.warn("PIT server response missing 'shards' field (server pre-8.16 compatibility fallback). Extracted id from raw body successfully.");
                return idNode.asText();
            }
        }
    }
```

KVK

Questions